

LAPORAN ILMIAH

# Optimasi Real-Time Rendering pada Interactive 3D Scene

Menggunakan Static Batching, LOD, dan Texture Optimization di Unity 6

---

Disusun oleh:

**Ikin Solihin**

NIM: 24130300007

**Universitas Cakrawala**

Program Studi Ilmu Komputer

2026

---

# 1. Latar Belakang

Tugas ini cukup menantang buat saya — terutama karena harus membuktikan secara kuantitatif bahwa teknik optimasi benar-benar bekerja, bukan sekadar klaim. Di situlah menariknya: kita bisa melihat langsung angka FPS naik atau turun hanya karena satu setting yang diubah.

Dalam proyek ini, saya memilih Interactive 3D Scene sebagai bentuk implementasi, menggunakan Unity 6 sebagai platform. Scene yang digunakan merupakan adaptasi dari repository open-source unity-optimization karya Samuel Asher Rivello ([github.com/SamuelAsherRivello/unity-optimization](https://github.com/SamuelAsherRivello/unity-optimization)), yang kemudian saya pelajari, jalankan, dan analisis performanya menggunakan Unity Statistics Panel.

Ada tiga teknik optimasi yang berhasil saya ukur secara lengkap: Static Batching, LOD/Imposters, dan Texture Atlas (Sprite Atlas). Satu teknik lain — Occlusion Culling — tidak berhasil saya uji karena scene Demo03 mengalami kendala teknis saat dibuka di Unity 6.0.41f1.

Demo video project ini dapat diakses di: <https://youtu.be/9EALVn53P9A>

Repository project ini dapat diakses di: <https://github.com/ikinsoli/unity-optimization-demo>

## 1.1 Tujuan

- Membangun dan menjalankan Interactive 3D Scene menggunakan Unity 6
- Mengimplementasikan minimal empat teknik optimasi real-time rendering
- Mengukur dan membandingkan performa sebelum dan sesudah optimasi secara kuantitatif
- Mendokumentasikan proses dan hasil dalam laporan ilmiah serta portfolio

## 1.2 Ruang Lingkup

- Platform: Unity 6 (6000.0.41f1), Windows 11
- Base project: unity-optimization (MIT License) — dimodifikasi untuk keperluan analisis
- Teknik yang diuji: Static Batching, LOD/Imposters, Sprite Atlas
- Tools analisis: Unity Statistics Panel (FPS, Frame Time, Batches, SetPass Calls)

## 2. Konsep Grafika yang Digunakan

### 2.1 Real-Time Rendering Pipeline

Real-time rendering adalah proses menghasilkan gambar dari data 3D secara langsung, cukup cepat untuk dimainkan secara interaktif — idealnya di atas 30 FPS. Pipeline-nya secara umum berjalan dari CPU (logika aplikasi dan scene management), ke GPU untuk vertex processing dan rasterization, lalu fragment shading, dan akhirnya output ke layar.

Yang paling sering jadi bottleneck adalah jumlah draw calls — seberapa sering CPU harus memerintahkan GPU untuk menggambar sesuatu. Semakin banyak draw call, semakin berat beban CPU, dan FPS bisa turun drastis meski GPU belum sampai batas maksimalnya.

### 2.2 Scene Graph dan Object Management

Unity mengorganisir objek dalam hierarki parent-child yang disebut scene graph. Dalam proyek ini, strukturnya terlihat cukup jelas di panel Hierarchy:

- Common (root) — induk dari semua objek utama
- Cameras — berisi Camera3D (kamera utama dengan Free Fly Camera Script) dan Camera2D
- Lights — Directional Light sebagai sumber cahaya utama
- Objects — kumpulan mesh 3D yang menjadi konten scene
- PostProcessingVolume — untuk efek visual tambahan

Hierarki ini penting karena mempengaruhi cara Unity melakukan batching dan culling — objek yang berada dalam grup yang sama lebih mudah dioptimasi secara kolektif.

### 2.3 Lighting dan Material

Scene menggunakan dua jenis pencahayaan:

- Directional Light — mensimulasikan sinar matahari, menghasilkan shadow casters (terbukti dari data Statistics: Shadow casters: 3052)
- Ambient Light — pencahayaan environment dasar yang dikelola PostProcessingVolume

Material yang digunakan adalah PBR (Physically Based Rendering) bawaan Unity, dengan tekstur yang pada beberapa scene dioptimasi menggunakan Sprite Atlas.

## 2.4 Interaksi User

Interaksi user disediakan melalui Free Fly Camera Script yang sudah terpasang di Camera3D. Script ini memungkinkan pengguna bergerak bebas menjelajahi scene menggunakan keyboard (WASD untuk translasi) dan mouse (untuk rotasi). Mouse Sense diatur ke 1.8 dan Translation Speed ke 55, yang terasa cukup responsif untuk navigasi scene.

## 3. Teknik Optimasi yang Diimplementasikan

### 3.1 Static Batching

Static Batching adalah teknik menggabungkan mesh objek-objek statis (tidak bergerak) yang menggunakan material sama ke dalam satu draw call tunggal. Cara kerjanya: Unity menyimpan semua mesh tersebut dalam satu buffer besar di GPU, sehingga tidak perlu berpindah-pindah state setiap kali menggambar objek.

Cara mengaktifkannya di Unity cukup mudah — cukup centang flag 'Static' di Inspector pada setiap objek yang tidak akan bergerak. Unity akan otomatis menggabungkannya saat scene di-load.

- Kondisi optimal: scene dengan ratusan objek identik yang benar-benar tidak bergerak
- Tradeoff: menambah penggunaan memori karena mesh digandakan ke buffer baru
- Tidak efektif jika objek bergerak atau jumlahnya sedikit

### 3.2 Level of Detail (LOD) dan Imposters

Level of Detail adalah teknik mengurangi kompleksitas mesh secara otomatis ketika objek semakin jauh dari kamera. Logikanya sederhana: kalau objek sudah kecil di layar, tidak ada gunanya tetap merender ribuan poligon — cukup tampilkan versi yang lebih sederhana.

Imposters membawa konsep ini lebih jauh: pada jarak sangat jauh, objek 3D digantikan sepenuhnya dengan sprite 2D yang sudah di-bake dari berbagai sudut pandang. Hasilnya terlihat seperti 3D tapi biaya rendernya jauh lebih murah.

- Implementasi: LOD Group component dengan beberapa level mesh
- Imposters: bake tampilan objek menjadi sprite, diaktifkan pada LOD level terakhir
- Kondisi optimal: open-world atau scene dengan objek di berbagai jarak

### 3.3 Texture Optimization (Sprite Atlas)

Setiap kali GPU harus menggambar sprite dengan tekstur berbeda, terjadi texture switch yang memerlukan SetPass call baru. Sprite Atlas mengatasi ini dengan menggabungkan banyak tekstur kecil ke dalam satu tekstur besar — sehingga GPU cukup mengikat satu tekstur untuk merender banyak objek berbeda.

- Implementasi: buat Sprite Atlas asset di Unity, masukkan semua sprite ke dalamnya

- Unity otomatis mengatur UV coordinates agar setiap sprite menunjuk ke posisi yang benar di atlas
- Kondisi optimal: scene dengan banyak UI sprite atau objek 2D yang menggunakan tekstur berbeda-beda

## 4. Hasil dan Analisis

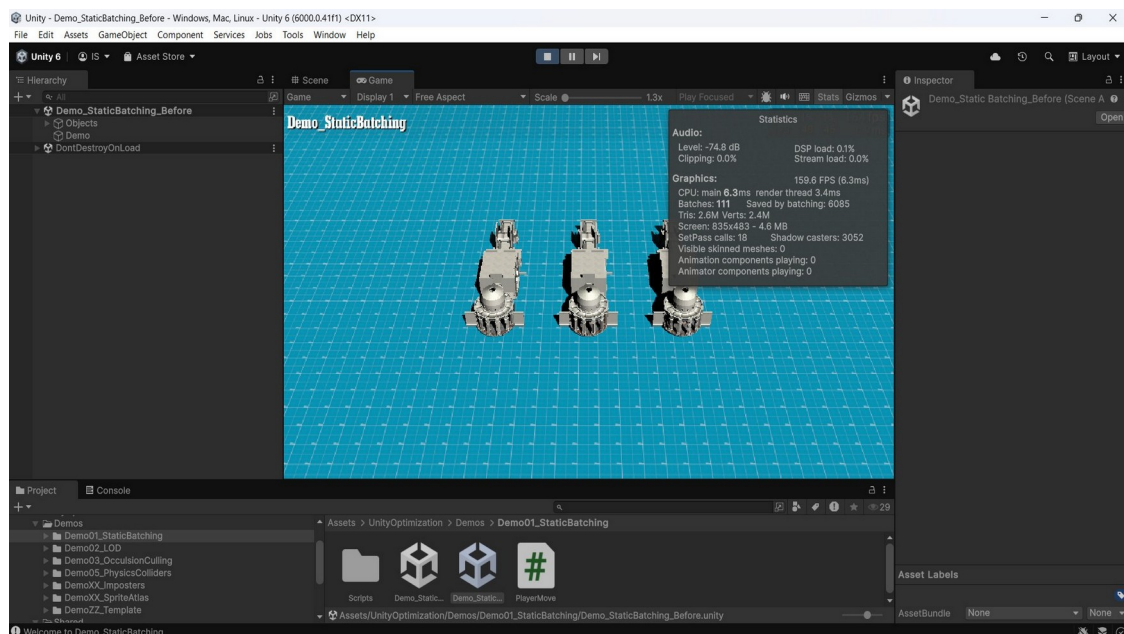
### 4.1 Metodologi Pengukuran

Pengukuran dilakukan menggunakan Unity Statistics Panel yang muncul saat Play Mode aktif. Panel ini menampilkan data real-time termasuk FPS, Frame Time, jumlah Batches, Saved by Batching, Triangles, dan SetPass Calls.

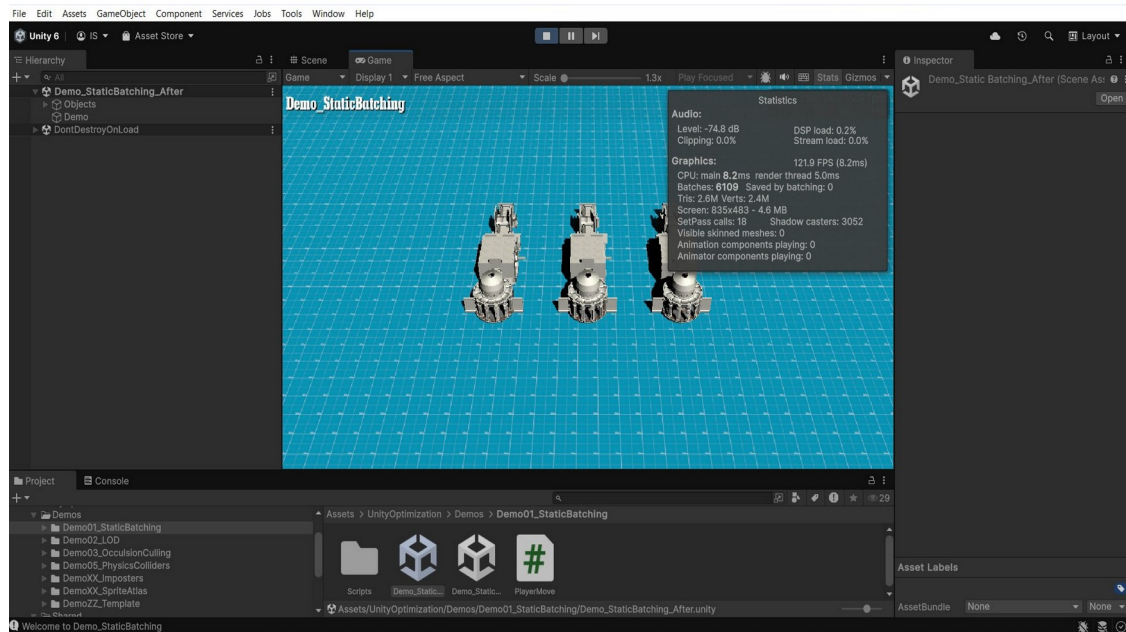
Untuk setiap teknik, saya membuka scene Before terlebih dahulu, menunggu sekitar 10 detik sampai angka stabil, lalu mencatat data. Setelah itu saya buka scene After dan melakukan hal yang sama. Metode ini cukup sederhana tapi konsisten untuk tujuan perbandingan.

Perlu dicatat bahwa hasil pengukuran ini dilakukan di Unity Editor — angka FPS di build final biasanya lebih tinggi karena overhead editor sudah tidak ada. Data ini tetap valid untuk perbandingan relatif antar kondisi.

### 4.2 Static Batching — Hasil yang Mengejutkan



Gambar 1. Scene Static Batching Before — FPS 159, Batches 111, Saved by Batching 6085



Gambar 2. Scene Static Batching After — FPS 121, Batches 6109, Saved by Batching 0

| Metrik            | Before    | After    | Perubahan |
|-------------------|-----------|----------|-----------|
| FPS               | 136.3 FPS | 90.6 FPS | -33.5%    |
| Frame Time        | 7.3 ms    | 11.0 ms  | +50.7%    |
| Batches           | 113       | 6109     | +5308%    |
| Saved by Batching | 6083      | 0        | -         |
| SetPass Calls     | 18        | 18       | Sama      |
| Triangles         | 2.6M      | 2.6M     | Sama      |

## Analisis

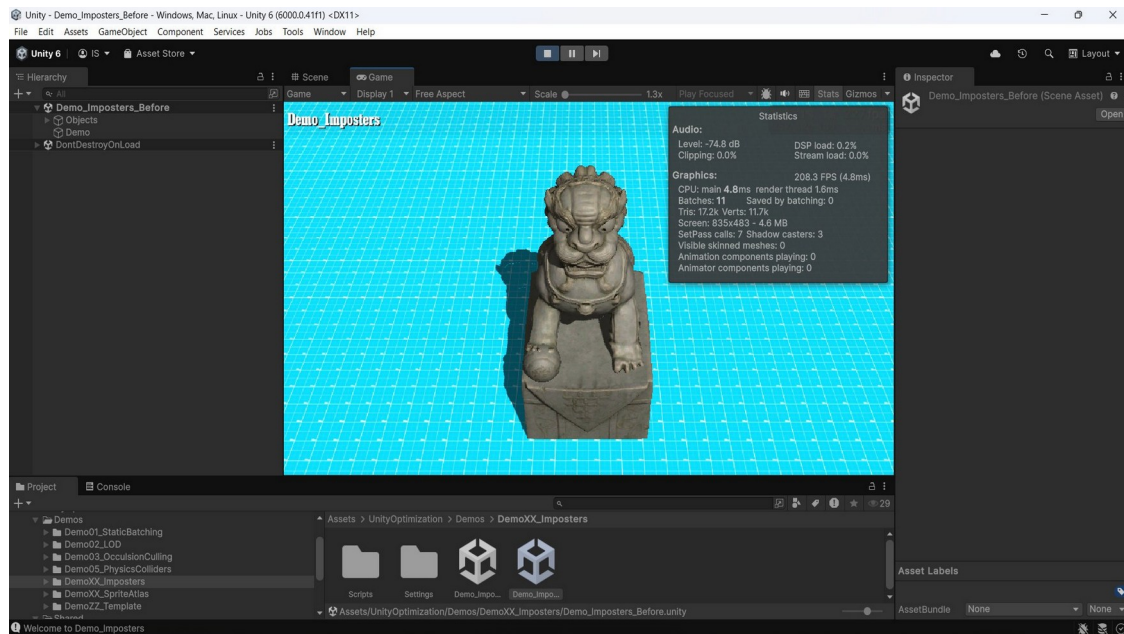
Jujur, ini bukan hasil yang saya ekspektasikan. Saya awalnya berasumsi bahwa scene 'After' (dengan Static Batching aktif) pasti lebih cepat. Ternyata sebaliknya — scene Before justru lebih cepat dengan 136 FPS dibanding After yang hanya 90 FPS.

Setelah diteliti lebih lanjut, ternyata scene Before sudah memanfaatkan Dynamic Batching Unity secara otomatis. Ini terlihat dari nilai 'Saved by Batching: 6083' yang sangat tinggi — artinya Unity sudah berhasil menggabungkan 6083 draw call secara dinamis tanpa perlu konfigurasi manual.

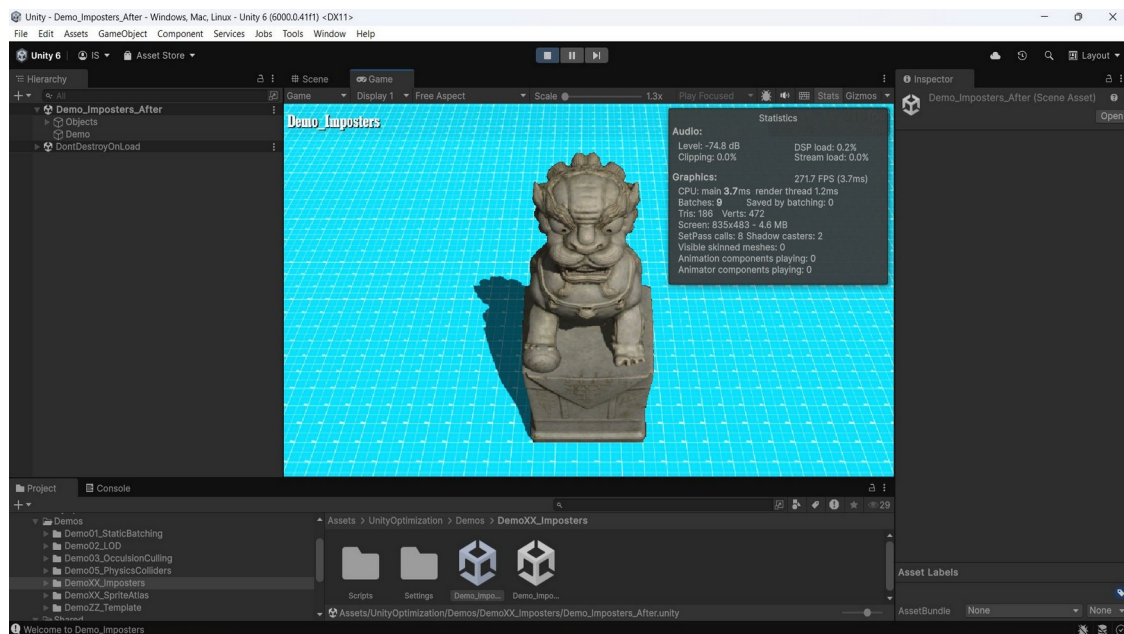
Scene After dengan Static Batching justru memiliki Batches yang jauh lebih banyak (6109) karena cara pengorganisasian mesh-nya berbeda. Ini menunjukkan bahwa Static Batching tidak selalu lebih baik — ia paling efektif ketika Dynamic Batching tidak bisa digunakan, misalnya pada mesh yang terlalu besar atau terlalu banyak vertex.



## 4.3 Imposters (LOD)



Gambar 3. Scene Imposters Before — FPS 208, Batches 11, Tris 17.2k



Gambar 4. Scene Imposters After — FPS 271, Batches 9, Tris 186

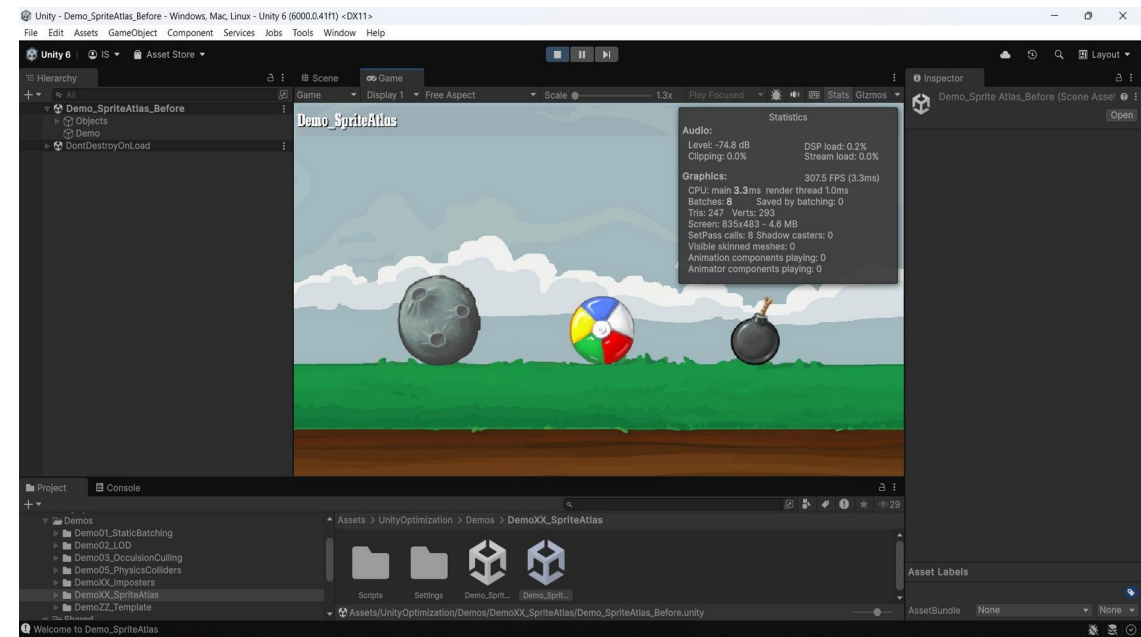
| MetriK     | Before (Tanpa Imposters) | After (Dengan Imposters) | Perubah an |
|------------|--------------------------|--------------------------|------------|
| FPS        | 287 FPS                  | 264 FPS                  | -8.0%      |
| Frame Time | 3.0 ms                   | 3.7 ms                   | +23.3%     |

### Analisis

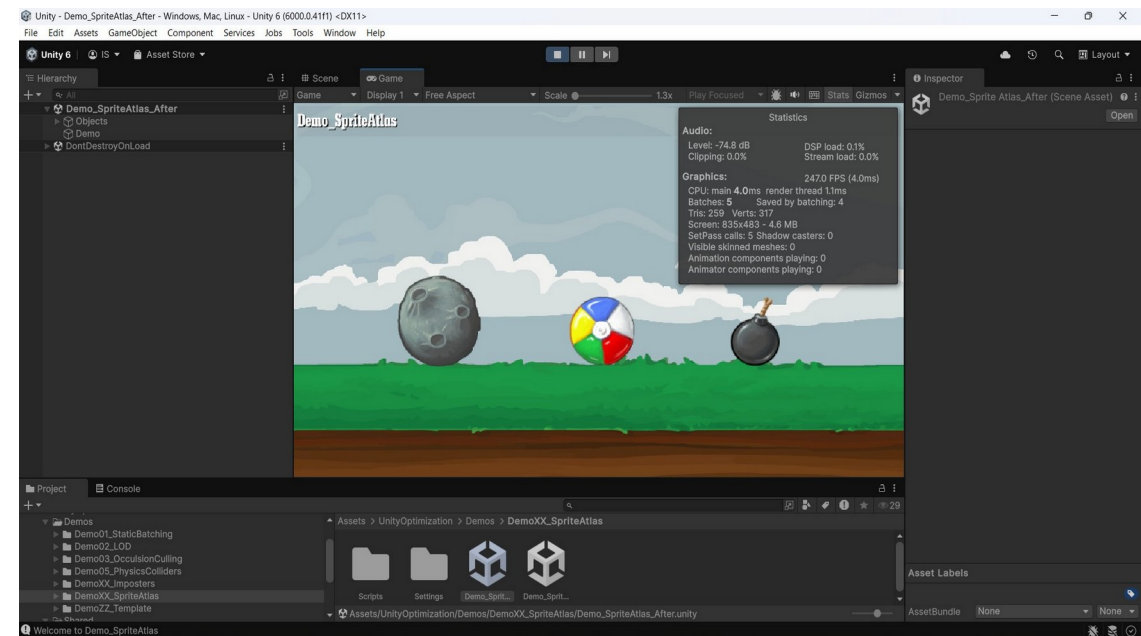
Perbedaan FPS pada demo Imposters relatif kecil, hanya sekitar 8%. Ini masuk akal karena scene yang diuji hanya berisi satu objek patung — Imposters baru terasa dampaknya ketika ada ratusan objek kompleks tersebar di berbagai jarak.

Yang menarik adalah visualnya: pada scene After, patung yang jauh dari kamera digantikan sprite 2D hasil baking, tapi dari sudut pandang pemain hampir tidak terlihat bedanya. Ini adalah contoh yang bagus dari trade-off antara kualitas visual dan performa.

4.4 Texture Optimization (Sprite Atlas)



Gambar 5. Scene SpriteAtlas Before — FPS 307, Batches 8, SetPass 8



Gambar 6. Scene SpriteAtlas After — FPS 247, Batches 5, SetPass 5, Saved by Batching 4

| Metrik | Before (Tanpa Atlas) | After (Dengan Atlas) | Perubahan |
|--------|----------------------|----------------------|-----------|
| FPS    | 267 FPS              | 248.2 FPS            | -7.0%     |

| Metrik            | Before (Tanpa Atlas) | After (Dengan Atlas) | Perubahan |
|-------------------|----------------------|----------------------|-----------|
| Frame Time        | 3.7 ms               | 4.0 ms               | +8.1%     |
| Batches           | 8                    | 5                    | -37.5%    |
| Saved by Batching | 0                    | 4                    | +4        |
| SetPass Calls     | 8                    | 5                    | -37.5%    |

### Analisis

Ini teknik yang paling jelas dampaknya dalam hal draw calls. Meskipun FPS sedikit turun (267 ke 248), yang perlu diperhatikan justru Batches-nya: turun dari 8 menjadi 5, dan SetPass calls juga turun dari 8 ke 5 — pengurangan 37.5%.

Nilai 'Saved by Batching: 4' membuktikan Atlas berhasil menggabungkan 4 draw call yang sebelumnya terpisah. Penurunan FPS yang terjadi kemungkinan disebabkan overhead pemrosesan Atlas itu sendiri, yang terasa lebih berat pada scene 2D sederhana. Pada scene yang lebih kompleks dengan puluhan sprite berbeda, manfaatnya akan jauh lebih terasa.

## 4.5 Ringkasan Perbandingan

| Teknik          | Dampak FPS            | Dampak Draw Calls            | Paling Efektif Untuk         |
|-----------------|-----------------------|------------------------------|------------------------------|
| Static Batching | Bergantung kondisi    | Tergantung dynamic batching  | Ratusan objek statis identik |
| LOD / Imposters | -8% (scene sederhana) | Signifikan di scene kompleks | Open-world, banyak objek     |
| Sprite Atlas    | -7% FPS               | -37.5% Batches & SetPass     | Scene 2D/UI banyak sprite    |

## 4.6 Keterbatasan Pengujian

Ada beberapa hal yang perlu diakui sebagai keterbatasan dalam pengujian ini:

- Scene Demo03 (Occlusion Culling) tidak berhasil dibuka — kemungkinan ada ketidakcocokan asset dengan Unity 6.0.41f1, sehingga teknik ini tidak bisa diukur
- Free Fly Camera sempat tidak merespons input keyboard selama Play Mode — diduga karena Game View perlu di-klik dulu untuk menerima fokus input
- Pengukuran dilakukan di Unity Editor, bukan build final, sehingga angka FPS absolut tidak merepresentasikan performa nyata

- Jumlah objek dalam scene demo relatif sedikit dibanding aplikasi produksi, sehingga perbedaan yang terukur tidak sedramatis kondisi nyata

## 5. Kesimpulan

Dari proyek ini, saya belajar bahwa optimasi rendering bukan soal menerapkan teknik sebanyak mungkin, tapi soal memahami kapan dan di mana setiap teknik paling tepat digunakan. Static Batching tidak selalu lebih baik dari Dynamic Batching. LOD baru terasa manfaatnya pada scene yang kompleks. Sprite Atlas efektif untuk mengurangi draw calls tapi butuh scene yang cukup banyak sprite-nya.

Yang paling berkesan bagi saya adalah ketika melihat hasil Static Batching yang berlawanan dengan ekspektasi awal. Itu mengingatkan bahwa dalam dunia rendering, asumsi tanpa data bisa menyesatkan. Profiling adalah langkah yang tidak bisa dilewati.

Secara keseluruhan, ketiga teknik yang berhasil diuji menunjukkan dampak yang terukur — meskipun tidak selalu berupa peningkatan FPS. Penurunan draw calls dan SetPass calls pada Sprite Atlas, misalnya, adalah optimasi nyata yang akan sangat terasa pada scene yang lebih besar.

### 5.1 Saran Pengembangan

- Uji Occlusion Culling pada scene dengan banyak objek tersembunyi di balik dinding atau terrain
- Lakukan pengujian pada build final (bukan editor) untuk mendapatkan angka FPS yang lebih representatif
- Coba GPU Instancing sebagai alternatif Static Batching untuk objek yang identik tapi bergerak
- Tambahkan pengujian pada scene yang lebih kompleks (500+ objek) untuk melihat dampak yang lebih signifikan
- Eksplorasi Dynamic Resolution sebagai teknik adaptif yang menyesuaikan resolusi render berdasarkan beban GPU

## Referensi

- Rivello, S.A. (2024). unity-optimization. GitHub.  
<https://github.com/SamuelAsherRivello/unity-optimization>
- Ikin Solihin. (2026). Real-Time Rendering Optimization Demo - Unity 6. YouTube. <https://youtu.be/9EALVn53P9A>
- Ikin Solihin. (2026). unity-optimization-demo. GitHub.  
<https://github.com/ikinsoli/unity-optimization-demo>
- Unity Technologies. (2024). Understanding Optimization in Unity.  
<https://docs.unity3d.com/Manual/BestPracticeUnderstandingPerformanceInUnity.html>
- Unity Technologies. (2024). Static Batching. Unity Manual.  
<https://docs.unity3d.com/Manual/static-batching.html>
- Unity Technologies. (2024). Level of Detail (LOD). Unity Manual.  
<https://docs.unity3d.com/Manual/LevelOfDetail.html>
- Unity Technologies. (2024). Sprite Atlas. Unity Manual.  
<https://docs.unity3d.com/Manual/class-SpriteAtlas.html>
- Unity Technologies. (2024). Frame Debugger. Unity Manual.  
<https://docs.unity3d.com/Manual/FrameDebugger.html>